

Contents

RVSPOC 2026 复现验证材料·蓝芯 LX5000 卷 1

三题判定速览 1

 为什么 S2603 与 S2604 无人达标 2

 另一道门槛：S2604 的覆盖率要求在数学上不可达 2

第一部分 编译与正确性 2

第二部分 性能与边界带复核 7

第三部分 S2605 make check 与失败归因 11

第四部分 热点函数表 15

第五部分 组委会标量基线 19

RVSPOC 2026 复现验证材料·蓝芯 LX5000 卷

版本 v2.3 发布日期 2026-09-29 编制：组委会验证侧

覆盖赛题：S2603 Redis、S2604 Memcached、S2605 RocksDB，共 11 个有效提交

本卷为详细验证记录；终裁结论见《总览与裁定卷》第八章——S2603/S2604 无人达标、S2605 冠军 #3

三题判定速览

本卷覆盖的三道赛题，均无提交达到赛题公开条款所定的性能门槛。S2603 与 S2604 无人达标、无冠军；S2605 经与赞助方共同评审，按「最接近达标、综合质量最优」评定 #3 为冠军（终裁见《总览与裁定卷》第八章）。

赛题	有无冠军	最好成绩	门槛	差距
S2603 Redis	无	+19.1% (#4, pipeline=64)	+30%	差 10.9 个百分点
S2604 Memcached	无	+3.6% (#1, base)	+30%	差 26.4 个百分点
S2605 RocksDB	冠军 #3 (经与赞助方共同评审)	seekrandom +31.7%、readrandom +29.9%~+32.6%、fillrandom +4.6% (均为 #3)	必测三项全部 ≥+30%	严格口径下无提交达标；#3 为最接近达标者（两项超门槛），按「最接近达标、综合质量最优」评定为冠军（终裁见《总览与裁定卷》第八章）

门槛出处为赛前公开的赛题详情页，非事后补充：

- <https://rvspoc.org/S2603>：「相比于 memtier_benchmark 标量版本在验证平台的基线测试结果，性能需提升至少 30%」
- <https://rvspoc.org/S2604>：同上
- <https://rvspoc.org/S2605>：「相比于 db_bench 标量版本在验证平台的基线测试结果，性能需提升至少 30%」

依据《验收备忘录》§ 5.4:「未过硬门槛者不参与排名」。即三题并非「矮子里拔高个」,而是无人具备参与排名的资格。

为什么 S2603 与 S2604 无人达标

并非选手实现质量问题,而是题目负载特性所致。两题的差距性质并不相同:

S2604 距门槛最远 (26.4 个百分点), 且差距均匀。五个场景的提升在 -0.6% 至 +3.6% 之间,没有任何一个场景接近门槛。热点采样给出了解释:该题首位热点为 pthread_mutex_lock, 占 14.26%; 锁原语合计 25.4 个百分点、系统调用合计 19.7 个百分点,而自有代码合计仅 17.8 个百分点。瓶颈在锁竞争与系统调用,向量化无从着力——即便把全部自有代码向量化,可优化的时间占比也不足两成。

S2603 差距较近 (10.9 个百分点), 且差距高度集中。八个场景中,六个非流水线场景仅 +0.5% 至 +4.4%,而两个流水线场景达 +12.1% 与 +19.1%。这一分布本身就是解释:流水线摊薄了系统调用开销,协议解析在时间占比中上升,向量化才得以发挥。#4 是四个提交中唯一向量化 RESP 协议扫描者,其增益恰好集中于此。

即:在向量化最能发挥作用的场景下,由唯一命中协议解析热点的实现取得的最好成绩仍只有 +19.1%,须在此基础上再提升 9.2% 方可达标。这是该门槛在本题不可达的直接证据,强于单纯的理论推演。

另一道门槛: S2604 的覆盖率要求在数学上不可达

S2604 详情页另载「现有 Memcached 核心函数以及内存分配清零函数的 RVV 向量化占比 >=70%」。而按 § 5.2 取 Top 20 热点为分母,该题 Top 20 中自有代码仅 8 项,占 40%——即便选手将全部自有代码向量化,覆盖率上限亦为 40%,永远无法达到 70%。

该题因此会在性能与覆盖率两道门槛上各失败一次。分母口径已按字面口径留档(不影响无人达标结论)(见《总览与裁定卷》第二部分 2.2),但无论采何种口径,均不改变本题无人达标的结论。

第一部分 编译与正确性

版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

生成日期: 2026-09-07 对应验收备忘录 § 4.B (可复现性)、§ 4.C (正确性)

覆盖 11 个有效提交的编译与功能正确性验证。性能测试 (§ 4.D) 另行安排。

归因原则: 每一项失败都区分「选手侧问题」与「组委会环境/操作问题」。所有正确性测试均设**标量对照组**(上游原版在同一板卡、同一工具链下跑同一套测试),只有相对对照组**新增**的失败才归因于选手改动。

一、结论总览

赛题	PR	编译	正确性	归因
S2603 Redis	#2	✗ LTO 冲突	未测	环境/构建配置, 代码本身可编译 (见 § 3.1)
S2603 Redis	#3	✓	✓ 与对照组完全一致	—

S2603 Redis	#4	✓	✗ 15 项失败	选手侧（见 § 3.2）
S2603 Redis	#6	✗ LTO 冲突	未测	环境/构建配置，代码本身可编译（见 § 3.1）
S2604 Memcached	#1	✓（需 gcc-14）	✓ testapp + make test 均通过	—
S2604 Memcached	#2	✓	✓ 唯一失败为构建产物所致（见 § 4.1）	—
S2604 Memcached	#3	✓	✓ 完整构建后通过（见 § 4.2）	—
S2605 RocksDB	#1	✓	✗ db_test 崩溃	选手侧（见 § 3.3）
S2605 RocksDB	#2	✓	✓ 零失败（优于对照组）	—
S2605 RocksDB	#3	✓	✓ 无新增失败	—
S2605 RocksDB	#4	✓（需 gcc-14 + PORTABLE）	✓ 无新增失败	—

编译：11 个提交中 9 个通过；2 个（S2603 #2/#6）因 LTO 与向量扩展冲突失败，但代码本身无误。

正确性：9 个受测提交中 7 个通过；2 个存在选手侧问题（S2603 #4、S2605 #1）。

二、验证环境与方法

- 平台：蓝芯 LX5000（32 hart, Ubuntu 24.04.4, 内核 6.18.3+），详见 本卷第五部分 § 1
- 编译器：gcc 13.3.0（默认）/ **gcc 14.2.0**（部分提交必需，见下）
- 构建命令：**逐个取自选手说明文件或 PR 描述**；来源标注为 doc、doc-adapted、default，用 default（选手文档未给明确命令）者失败时不直接归因于选手
- 正确性套件：Redis runtest --accurate --no-latency、Memcached testapp + make test、RocksDB db_test（LIB_MODE=static）
- 标量对照组：上游 Redis 8.8.0 / Memcached 1.6.42 / RocksDB v11.1.1，同板同工具链

对照组基准

对照组	结果
Redis 8.8.0	5552 通过 / 0 失败
Memcached 1.6.42	testapp 通过；make test 通过（exit=0，无 not-ok）
RocksDB v11.1.1	1 失败：DBTest.PurgeInfoLogs（上游固有）

三、选手侧问题（需进公开讨论流程）

3.1 S2603 #2、#6 — LTO 与向量扩展冲突【性质：构建配置，非代码缺陷】

现象：gcc-13 与 gcc-14 均失败于 lto1: fatal error: target specific builtin not available。

归因实验：仅追加 -fno-lto、其余不变，**两者均编译通过**（results/lto_experiment.txt）。

实验修正（2026-09-10）：首轮实验对 #6 的重建命令漏传了其文档要求的 BUILD_RVV=yes MALLOC=libc BUILD_TLS=no——该仓库的 RVV 对象（rvv_optim.o）仅在 BUILD_RVV=yes 时参与编译，故首轮「#6 加 -fno-lto 可编过」实为无 RVV 的空验证。已按选手文档完整开关补做（results/lto_experiment_pr6.txt）：**带 BUILD_RVV=yes 时追加 -fno-lto 依然编译通过**，结论修正后仍然成立。#2 无文档构建命令（default），首轮实验命令即有效，无需修正。故选手的 RVV 代码本身正确，问题在于 Redis 默认 -flto=auto：LTO 在链接期以基线 -march 重新生成代码，V 扩展 builtin 随之失效。

参照：S2603 #3 的说明文件预见了此问题，并给 RVV 对象单独加 -fno-lto 隔离，构建正常。

建议处置：非代码缺陷，宜按 § 4.B 公开讨论，请选手说明其原始构建环境下的 LTO 处理方式。是否允许组委会加 -fno-lto 后继续评审，请评判组定夺。

3.2 S2603 #4 — 15 项测试失败【性质：选手侧】

对照组 0 失败，本提交 15 失败，分两类：

(a) 9 项 benchmark 集成测试——已定位到确切成因：

选手在 src/monotonic.c:187 新增了 RISC-V 分支（上游仅有 x86 第 87 行、aarch64 第 130 行），该分支无法确定时钟频率，于是**每次启动向 stderr 无条件打印** monotonic: riscv, unable to determine clock rate。

Redis 测试用 catch { exec {*} \$cmd } 调用 redis-benchmark，而 **Tcl 的 exec 只要子进程写 stderr 就抛错**（即便退出码为 0），于是全部 benchmark 测试判失败。

决定性对照（同一服务器、同一命令、同一 Tcl 调用）：

二进制	Tcl exec
选手 #4 的 redis-benchmark	抛错
对照组的 redis-benchmark	正常

注：redis-benchmark 单独运行功能正常（退出码 0、结果正确），影响仅限于破坏上游测试套件。

(b) 6 项功能测试——4 项 Prefetch、1 项 MEMORY USAGE、1 项 reply buffer limits。已通过隔离实验定位（results/phase8.txt）：

将选手的 networking.c/server.c 恢复为上游版本后重测，**15 项失败一项都未消失——失败与协议解析路径的改动无关，根因在 src/memory_prefetch.c：**

选手在 dictPrefetchKeys() 与 prefetchCommands() 入口加了条件编译：

```
#if defined(__riscv) && !defined(__riscv_zicbop)
    return; /* 整个 Prefetch 功能被禁用 */
#endif
```

__riscv_zicbop 是**编译器宏**，仅当 -march 显式包含 zicbop 时定义，与 CPU 是否实现该扩展无关。本提交构建 -march=rv64gcv（不含 zicbop），故宏未定义 → return 生效 → **Prefetch 功能整体被禁用**。而测试断言 io_threaded_total_prefetch_entries 在 2-15 之间，禁用后计数恒为 0，4 项 Prefetch 测试必然失败；assert_range 的注释还特意写了「With slower machines, the number of prefetch entries can be lower」——即上游设计允许慢，不允许没有。

事实层面：LX5000 实测 ISA 包含 `_zicbop`（缓存预取扩展，CPU 支持），但构建标志未通告该扩展，故功能被编译期条件误杀。MEMORY USAGE 与 reply buffer limits 两项与 `USE_PROCESSOR_CLOCK`（选手在 Makefile 中新增的宏，改用处理器时钟）相关，同属编译配置引发的测试预期偏差。

性质：非算法/数据结构错误——所有失败源于「构建配置与特性探测」：以编译期宏替代运行时探测，导致 CPU 明明支持的特性被禁用。是否构成正确性不达标，请评判组结合公开讨论的选手答复后定夺。

3.3 S2605 #1 — db_test 崩溃：使用了验证平台不支持的 Zbc 指令【性质：选手侧】

现象：DBTest.MockEnvTest 收到 signal 4（非法指令），core dump。

崩溃点（gdb 符号化定位）：

```
util/crc32c_riscv.cc:66  CrcCmulLong  ->  uint64_t tmp = Cmul(msgpoly, kBarrett1);
出错指令: cmul a5,a5,a4
调用链: crc32c_riscv -> CrcCmul -> CrcCmulUpdatePartial -> CrcCmulLong
上层: log::Writer::Writer -> DBImpl::NewDB -> DBImpl::Recover -> DB::Open
```

根因：`cmul` 属 **Zbc**（标量无进位乘法）扩展，而 LX5000 不支持 **Zbc**：

板卡实测 ISA	结果
标量 bitmanip	zba、zbb、zbs（无 zbc）
向量	zvbb、zvbc（有向量版 cmul）

选手新增文件 `util/crc32c_riscv.cc` 以内联汇编 `.option arch,+zbc` 直接发射 `cmul / cmulh、cmulr`，绕过了编译器对 `-march` 的检查；且据其自身注释与代码，选择路径完全是编译期决定、无运行时检测：

- `crc32c_riscv.cc:27-28`：「HAVE_RISCV_CRC32C already guarantees the target supports Zbc」
- `crc32c.cc:1121`：「selection is compile-time (-march=...zbc...)」
- 实际构建标志：`-march=rv64gcv_zbc_zvbc`

即：构建期依据编译器是否接受 `zbc` 来启用该路径，而非目标 CPU 是否实现该扩展。LX5000 上编译器接受、CPU 不支持，于是首次写 WAL（`DB::Open` 路径上的 CRC32C）即崩溃。

隔离验证：同一代码用选手文档中的标量变体 `PORTABLE=rv64gc` 重编，MockEnvTest 通过（`exit=0`）；向量/Zbc 构建则崩溃。

与规则的关系：S2605 赛题明确规定「验证平台：蓝芯 LX5000 开发板」，产出要求含「能在验证平台正确运行 RocksDB 核心测试集（db_test）」。当前提交在验证平台无法运行。

背景（可作评判参考）：选手说明文件称「所有工作均在挑战指定的 Spacemit K3 开发板上构建和验证」。K3 支持 Zbc，故其本地验证不会暴露此问题。但赛题规则指定的验证平台是 LX5000，非 K3；该认知偏差是问题未被选手自查发现的原因。

建议处置：按 § 4.B 公开讨论，向选手说明崩溃点、出错指令与平台 ISA 差异，请其解释运行时检测的缺失。是否允许以标量回退路径重测、以及本项对「正确性达标」的影响，请评判组定夺。

四、经查证不归因于选手的失败

4.1 S2604 #2 — make test 中 whitespace 检查失败

失败项为 ar-lib (t/whitespace.t, 307 项中 1 项)。经核：ar-lib 是本地执行 autogen.sh 生成的符号链接 (→ /usr/share/automake-1.16/ar-lib)，未纳入版本控制，属构建产物而非选手提交内容。不归因于选手。

4.2 S2604 #3 — testapp 失败【组委会构建错误】

首轮组委会使用了简化命令 make memcached testapp，遗漏选手文档中的 memcached-debug 等目标，导致 testapp 无法 exec 到被测二进制。按选手文档完整目标 (make memcached memcached-debug testapp sizes timedrun rvv-test rvv-bench) 重建后，testapp 退出码 0、通过。组委会侧错误，已更正。

4.3 S2605 #3、#4 — DBTest.PurgeInfoLogs

与标量对照组的失败项完全相同，上游固有，不归因于选手。

4.3b S2605 全体 — range_locking_test 的 14 项 SIGILL (2026-09-13 make check 补测发现)

标量对照组的 make check 中，range_locking_test 14 个用例全部以 signal 4 崩溃。gdb 定位：toku_time.h:155 的内联汇编 rdcycle——Linux 自 6.6 起禁止用户态读取该寄存器 (本板内核 6.18.3+)。上游固有，不归因于选手；各提交的 make check 须以此为基准线扣除。详见《LX5000 卷》第三部分 § 1。

4.4 编译阶段的环境依赖 (3 个提交)

PR	缺失能力	处置
S2603 #2	RVV tuple 类型 intrinsics (__riscv_vsseg3e8_v_u8m2x3 等)	装 gcc-14 (gcc-13 无此支持)
S2604 #1	Zvbb intrinsics (__riscv_vro_l_vx_u32m8)	同上
S2605 #4	构建脚本面向交叉编译 (/opt/riscv 工具链)	改用等效原生参数 -DPORTABLE=rv64gcv

共性文档缺陷：上述提交的说明文件均未写明最低编译器版本或构建前置条件。规则要求说明文件包含「依赖库说明」与「编译步骤」，工具链版本属此列。建议记录为文档质量扣分项，不因此判不合格。

五、组委会侧的操作问题 (已修正，记录备查)

评审过程中发生并已修正的问题，一并留档以保证过程可审计：

问题	影响	修正
diff 基线误用首个 commit	提交仓库是上游完整 fork，55 个真实改动被算成 8022 个	改用 merge-base origin/HEAD
硬编码默认分支 main/master	Redis 默认分支为 unstable，4 个 PR 全部失败	改用 origin/HEAD 自动解析
只扫仓库文件、未读 PR 描述	S2604 #4 被误判为「无说明文件」	两处合并读取
逐 PR 独立克隆大仓库	链路中断导致 CLONE_FAIL	每仓库缓存一次，本地派生
git clone --shared 未断开对象依赖	工作树可能损坏	派生后 repack 并删 alternates

grep -c 与 \\ \\ echo 0 叠加	结果表列错位	改用 \${p:-0}
正确性脚本无幂等	触发全量重跑，且对照组守卫失效致 RocksDB 对照组无法补跑	改为逐项跳过
关键词计数当判定依据	曾误报对照组「47 项失败」，实为 0	改用测试框架自身结论 (^[err]:、退出码)
简化选手构建命令	S2604 #3 被误判失败	严格照选手文档全部目标
LTO 实验对 S2603 #6 漏传 BUILD_RVV=yes	RVV 代码未参与编译，「-fno-lto 可编过」为空验证	带全套文档开关重做，结论仍成立（见 § 3.1 修正注）

六、待办

1. S2605 #1 崩溃定位 **已完成**：见 § 3.3，定位至 `crc32c_riscv.cc:66` 的 `Zbc c1mul` 指令
2. S2603 #4 的 6 项功能测试失败 **已定位**：见 § 3.2(b)，编译期宏误禁 `Prefetch`
3. **公开讨论**：S2603 #2/#6 (LTO)、S2603 #4、S2605 #1 三项需按 § 4.B 在 PR 评论区公开现象并与选手讨论 —— **所有对外沟通须先经组委会审批后发出**。草稿已备于 `review/discussion_drafts.md` (内部文件，均未发出) (待审批)
4. **性能测试** (§ 4.D，7 个通过正确性验证的提交 + 2 个条件组)：**2026-09-10 完成**，结果见《LX5000 卷》第二部分。摘要：S2603/S2604 全部远低于 +30% 门槛 (± 个位数百分比)；S2605 #3 的 `seekrandom` +32.2% 达标、`readrandom` +29.2% 边界带复核中、`fillrandom` +4.7% 未达 (门槛口径待决策 D7)；S2605 #4 必测三项全部慢于基线；条件组 S2603 #6 开 AOF 即断言崩溃 (输入至 D3)

七、留档

内容	路径 (LX5000 ~/rvspoc-verify/)
编译状态	<code>results/build_status.tsv</code> 、 <code>results/build_retry.tsv</code>
正确性结果	<code>results/correctness.tsv</code>
归因实验	<code>results/lto_experiment.txt</code> 、 <code>results/phase6.txt</code> 、 <code>results/phase7.txt</code> 、 <code>results/pr4_recheck.txt</code>
全部构建与测试日志	<code>logs/</code> (每个提交每个套件独立日志)
脚本	<code>build_all.sh</code> 、 <code>retry_failed.sh</code> 、 <code>correctness.sh</code> 、 <code>phase*.sh</code>

第二部分 性能与边界带复核

版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

测试日期：2026-09-10 对应验收备忘录 § 4.D 基线依据：本卷第五部分 (唯一判定基准)

口径与组委会基线完全一致：同板、同 `memtier/db_bench` 参数、绑核 (server 0-7 / client 16-31)、每场景 **3 轮取中位数**、轮间冷却 30s、场景间 90s。原始数据：LX5000 ~/rvspoc-perf/results/。门槛：vs 组委会标量基线 $\geq +30\%$ (备忘录 § 4.D)；落在 +28%~+32% 边界带的按本卷第五部分 § 4 重跑复核后定论。

一、结论总览

赛题	PR	+30% 门槛	摘要
S2604	#1	✗ 未达	最好场景 +3.6% (base)
S2604	#2	✗ 未达	最好场景 +3.3% (base); 唯一开 <code>--enable-riscv-rvv</code> 的提交
S2604	#3	✗ 未达	最好场景 +2.2% (setheavy)
S2603	#3	✗ 未达	最好场景 +5.6% (base)
S2605	#2	✗ 未达	必测三项 -4.7% ~ +7.2%
S2605	#3	冠军 (严格口径无提交达标, 经与赞助方共同评审按最接近达标、综合质量最优评定)	seekrandom +31.7% 达标; readrandom +29.9%~+32.6%; fillrandom +4.6% 未达
S2605	#4	✗ 未达	必测三项全部慢于基线 (-6.7% ~ -18.2%)

条件组 (是否采信取决于待决策 D3, 数据仅备用):

赛题	PR	结果
S2603	#2 (-fno-lto 构建)	全场景 $\leq +0.1\%$, 多数为负
S2603	#6 (-fno-lto + BUILD_RVV=yes 构建)	无 AOF 场景最好 +16.5% (pipe64); AOF 场景服务端断言崩溃 (见 §4)

总体图景: S2603/S2604 两题 (网络服务型) 所有提交与基线差异都在个位数百分比, 远低于 +30%——QPS 瓶颈在网络/系统调用/事件循环路径, 字符串与协议解析的向量化收益被稀释。唯一逼近门槛的是 S2605 #3 (存储引擎内核路径 + O3/PGO 交付构建)。

二、详细数据 (Totals QPS 中位数; 括号内为 vs 基线)

S2604 Memcached

场景	基线	#1	#2	#3
base	109,621	113,514 (+3.6%)	113,253 (+3.3%)	110,426 (+0.7%)
highconc	150,285	149,328 (-0.6%)	146,219 (-2.7%)	149,655 (-0.4%)
setheavy	105,816	107,308 (+1.4%)	106,888 (+1.0%)	108,164 (+2.2%)
highval4k	103,932	104,185 (+0.2%)	103,610 (-0.3%)	105,217 (+1.2%)
pipe16	677,363	695,563 (+2.7%)	695,945 (+2.7%)	676,238 (-0.2%)

S2603 Redis (#3, 正式; #2/#6 见 §4 条件组)

场景	基线	#3
base	41,827	44,160 (+5.6%)
highconc	29,752	29,739 (-0.0%)

writeheavy	41,162	40,544 (-1.5%)
highval4k	42,048	42,384 (+0.8%)
highval16k	36,035	36,360 (+0.9%)
pipe16	412,904	420,924 (+1.9%)
pipe64	746,708	775,671 (+3.9%)
persist	41,524	41,965 (+1.1%)

S2605 RocksDB db_bench (ops/sec 中位数)

项目	基线	#2	#3	#4
fillrandom (必测)	244,774	262,344 (+7.2%)	256,195 (+4.7%)	228,350 (-6.7%)
readrandom (必测)	292,356	287,483 (-1.7%)	377,690 (+29.2%) △边界带	251,370 (-14.0%)
seekrandom (必测)	141,371	134,675 (-4.7%)	186,910 (+32.2%) ✓	115,603 (-18.2%)
fillseq (参考)	260,890	286,073 (+9.7%)	322,668 (+23.7%)	240,298 (-7.9%)
overwrite (参考)	243,093	257,581 (+6.0%)	260,901 (+7.3%)	225,586 (-7.2%)
readseq (参考)	8,887,222	9,203,964 (+3.6%)	12,007,378 (+35.1%)	7,566,544 (-14.9%)

构建说明（各按选手文档，doc-adapted 指以板载 gcc-14 等效替代选手自有工具链路径）：

- **#2**：CC/CXX=gcc-14, LIB_MODE=static DEBUG_LEVEL=0, EXTRA_C(XX)FLAGS=-march=rv64gcv_zba_zbb_zbc (doc-adapted)。march 含 zbc 但运行未见非法指令——其文档所称 hwprobe 运行时检测与实测行为一致。
- **#3**：按其 docs/REPRODUCE.md 交付配方**现场执行**：RVA23 子集 march (rv64gcv_zba_zbb_zbs_zicbop_zicnd) + O3 + instrument→三负载训练→profile-use 重建（现场新鲜 PGO，符合其「必须紧邻评测现场训练」的硬性要求；-j 并行度按板载调整，不影响代码生成）。
- **#4**：cmake -DPORTABLE=rv64gcv -DCMAKE_BUILD_TYPE=Release, gcc-14 (doc-adapted, 等效替代其交叉编译脚本)。**必测三项全面慢于基线**，与其自报数据方向相反；构建配置已忠实于文档，负收益本身即为板上实测结论。

S2605 #3 边界带复核（已完成）

readrandom +29.2% 落在 +28%~+32% 复核带（本卷第五部分 § 4），按原协议追加 3 轮 (results/s2605-pr3-recheck/)。两轮对比与六轮合并：

项目	首轮中位 (3 轮)	复核中位 (3 轮)	六轮合并中位	vs 基线
readrandom	377,690 (+29.2%)	387,537 (+32.6%)	379,869	+29.9%
seekrandom	186,910 (+32.2%)	185,610 (+31.3%)	186,260	+31.7%
fillrandom	256,195 (+4.7%)	255,846 (+4.5%)	256,020	+4.6%

结论：seekrandom 复核后稳定达标（两轮均 >31%）；fillrandom 稳定未达；**readrandom 复核后仍骑在线上**——首轮 +29.2%、复核轮 +32.6%、六轮合并 +29.9%，轮间波动（跨会话 ±2~3%，见基线报告 § 4）已大于其与门槛的距离。是取「六轮合并中位数」（+29.9%，未达）

还是「复核轮中位数」(+32.6%，达标)属判定口径，组委会未预先规定，不由测试方单方认定——已并入待决策 D7 一并裁定。无论 readrandom 判法如何，在「三项全部达标」口径下 #3 均不过硬门槛（fillrandom 明确未达）。

三、S2605 #3 的门槛口径问题（新增待决策 D7）

备忘录 § 4.D 写「db_bench 三项 OPS..... vs 标量基线 $\geq +30\%$ 」，未明确三项须全部达标、任一达标，还是加权/平均。#3 实测：seekrandom 达标 (+32.2%)、readrandom 边界带（复核中）、fillrandom 明显未达 (+4.7%)。三种口径给出三种结论，属规则解释，须评判组决定（DECISIONS_PENDING.md（内部文件，不随本次交付）D7）。其余提交在任何口径下结论不变。

四、条件组：S2603 #2 / #6（-fno-lto 构建，采信与否取决于 D3）

数据仅备查：两提交因 LTO 构建失败未过可复现性验证（待决策 D3）；为省板时一并采数。

场景	基线	#2 (-fno-lto)	#6 (-fno-lto + BUILD_RVV=yes)
base	41,827	40,954 (-2.1%)	42,905 (+2.6%)
highconc	29,752	29,553 (-0.7%)	29,852 (+0.3%)
writeheavy	41,162	40,654 (-1.2%)	41,481 (+0.8%)
highval4k	42,048	41,258 (-1.9%)	43,295 (+3.0%)
highval16k	36,035	36,054 (+0.1%)	36,558 (+1.5%)
pipe16	412,904	398,984 (-3.4%)	451,382 (+9.3%)
pipe64	746,708	687,497 (-7.9%)	870,273 (+16.5%)
persist	41,524	40,680 (-2.0%)	0 —— 服务端崩溃

#6 的 AOF 崩溃（results/s2603-pr6-CONDITIONAL/redis-server.log）：

Guru Meditation: You tried to retain an object allocated in the stack #object.c:637
调用链：readQueryFromClient → processCommand → call → alsoPropagate → incrRefCount

机制：开启 AOF 后命令须向 AOF 传播，alsoPropagate 会 retain 命令参数对象；#6 的改动使栈上分配的 robj 进入该路径，触发 Redis 的对象生命周期断言，服务端 panic。无持久化的 7 个场景全部正常——该缺陷只在 AOF 开启时暴露，恰是其未跑过回归测试（LTO 编译失败）而漏掉的问题。对 D3 的意义：即便允许 -fno-lto 续审，#6 也须先过正确性关（预期 runtest 的 AOF 相关用例会命中此崩溃）。注意此结论基于组委会 -fno-lto 构建；断言逻辑与 LTO 无关，归因于选手改动，但如进入正式流程宜在公开讨论中给选手解释机会。

五、留档

内容	路径（LX5000）
流水线与复核脚本	~/rvspoc-perf/perf_all.sh、recheck_pr3.sh
全部原始输出（每场景每轮独立文件）	~/rvspoc-perf/results/s260*/
汇总	~/rvspoc-perf/results/summary.tsv
S2605 db_bench 构建日志	~/rvspoc-perf/logs/

#6 崩溃现场	~/rvspoc-perf/results/s2603-pr6-CONDITIONAL/redis-server.log
运行日志	~/rvspoc-perf/pipeline.log、recheck.log

第三部分 S2605 make check 与失败归因

日期：2026-09-13（已完成） 对应验收备忘录 § 4.C 「make check、db_test 在 LX5000 通过」

结论总览

提交	测试阶段	相对对照组的 新增 失败	§ 4.C 正确性结论
对照组 v11.1.1	FAIL	—	基准线：range_locking_test 因上游 rdcycle SIGILL
#3	38,934 项全通过	无；且修复了对照组的上游失败	通过 （唯一未过项为 check-format，属工具缺失+代码规范）
#2	FAIL	无	通过 （两类失败均经实验证明非选手责任）
#4	FAIL	无	通过 （同上）
版本 v1.0 发布日期 2026-09-22 本包 17 份文件 共用同一版本号			

三个受测提交没有任何一项失败归因于选手。全部失败可归为两类，均已实验定性：range_locking_test（上游 rdcycle 缺陷）与 options_settable_test（gcc-14 codegen 写 padding）。

背景：复现验证阶段只执行了 db_test，遗漏了备忘录明文要求的 make check（全量单测，约 39,000 个用例）。本报告补齐该项。**方法：**沿用正确性阶段的原则——先跑**标量对照组**（上游 v11.1.1 + 发行版默认工具链），只有相对对照组**新增**的失败才归因于选手。执行顺序按评审价值：对照组 → #3 → #2 → #4。**#1 跳过：**其 DB::open 即因 Zbc 非法指令崩溃（见《LX5000 卷》第一部分 § 3.3），make check 无参考意义。

脚本 ~/rvspoc-makecheck.sh，日志 ~/rvspoc-makecheck/logs/，汇总 ~/rvspoc-makecheck/results/makecheck.tsv。

一、标量对照组（上游 RocksDB v11.1.1）—— 已完成

结果：make check 整体 FAIL，耗时 2,188 秒。**失败用例全部集中在 range_locking_test**，退出码一律 132 (= 128 + SIGILL，非法指令)，本次运行列出 14 项。

计数说明：make check 在失败后中止派发，故各次运行列出的失败条目数取决于中止时机（对照组列出 14 项、#4 列出 16 项），**不是真实差异**。判定以「失败集合是否落在 range_locking_test 之内」为准，不以条目数比大小。

失败清单：

用例	退出码
RangeLockingTest.BasicRangeLocking	132
RangeLockingTest.LockWaitCount	132
RangeLockingTest.LockWaiteeAccess	132
RangeLockingTest.MultipleTrxLockStatusData	132
RangeLockingTest.SnapshotValidation	132
RangeLockingTest.UpgradeLockAndGetConflict	132
AnyLockManagerTest.Deadlock/0	132
AnyLockManagerTest.GetWaitingTxns_MultipleTxns/0	132
AnyLockManagerTest.LockConflict/0	132
AnyLockManagerTest.LockDowngrade/0	132
AnyLockManagerTest.LockUpgrade/0	132
AnyLockManagerTest.ReentrantExclusiveLock/0	132
AnyLockManagerTest.ReentrantSharedLock/0	132
AnyLockManagerTest.SharedLocks/0	132

根因（gdb 符号化定位，非推测）：

```
#4 toku_time_now() at ../range_tree/lib/portability/toku_time.h:155
155  asm volatile("rdcycle %0" : "=r"(cycles));
#5 toku::locktree::sto_end_early (locktree.cc:285)
#6 toku::locktree::sto_try_acquire (locktree.cc:350)
#7 toku::locktree::acquire_lock (locktree.cc:521)
```

上游的 TokuDB 范围锁代码在 toku_time_now() 中直接内联汇编发射 **rdcycle**。Linux 自 6.6 起默认禁止用户态读取 **rdcycle**（本板内核 6.18.3+），故一执行即 SIGILL。

定性：上游固有缺陷，与本届任何选手无关。后续各提交的 make check 结果须以此 14 项为基准线扣除，只有新增失败才归因于选手。

一个值得记录的交叉印证

S2605 #3 的说明文件（docs/REPRODUCE.md）主动记载了这一问题及其修复：

```
toku_time.h: rdcycle → rdtime (Linux ≥6.6 用户态 rdcycle SIGILL, 修复 16 个
range_locking_test 失败)
```

选手独立发现并修复了同一缺陷，其描述（含触发条件「Linux ≥6.6」）与组委会实测完全吻合（选手记 16 项、组委会实测 14 项，差异应为版本或用例枚举口径，不影响定性）。这是一项真实的 riscv64 移植修复，建议在代码审查中作为加分项记录。其 make check 结果（下节）将同时验证该修复是否生效。

二、S2605 #3 —— 测试阶段全部通过【结论：正确性无问题】

测试阶段：38,934 个用例全部通过，零失败（parallel 进度跑满至 Left: 0，无任何非零 Exitval）。耗时 2,316 秒。

关键点：对照组挂掉的 14 项 `range_locking_test` 在本提交下全部通过——即选手文档所述的 `toku_time.h: rdcycle → rdttime` 移植修复经实测验证生效。这不仅是「无新增失败」，而是修复了上游在本平台的固有缺陷，建议代码审查记为加分项。

唯一的失败项：make check-format (Makefile:1332, Error 128) ——> You didn't have clang-format-diff.py and/or clang-format available in your computer!

归因：组委会环境缺件，非选手问题。板上未安装 clang-format，已于 2026-09-13 补装 (Ubuntu clang-format 18.1.3)。对照组未暴露此问题，是因为它在更早的测试阶段就已失败退出，根本没走到 check-format 这一步。

口径建议（另见下节）：check-format 是代码风格 lint，不是功能正确性检查。备忘录 § 4.C 的正确性判定应以测试阶段结论为准；clang-format 的结果归入 § 4.E 「代码规范，酌情记录」。否则会出现「选手代码全部测试通过、却因风格 lint 判正确性不达标」的荒谬结论。

三、S2605 #2 —— 3 项失败，其中 2 项确定不归因于选手，1 项待隔离实验定性
耗时 1,517 秒，测试跑满 (Left: 0)。失败三项：

用例	退出码	归因
range_locking_test-shard-0	132 (SIGILL)	上游固有（同对照组，rdcycle，见 § 1）
range_locking_test-shard-1	132 (SIGILL)	上游固有（同上）
options_settable_test-shard-0	1	待定性（见下）

OptionsSettableTest.ColumnFamilyOptionsAllFieldsSettable 的定性

```
options/options_settable_test.cc:730: Failure
Expected equality of these values:
  unset_bytes_base           Which is: 102
  NumUnsetBytes(new_options_ptr, sizeof(ColumnFamilyOptions), ...)
                               Which is: 126
```

该用例通过「统计结构体中未被写入的字节数」来验证所有字段可设置。结构体 padding 一旦被编译器写入（store-merging 优化），计数即偏移——与选手是否改动业务逻辑无关，而与编译器版本与 -march 直接相关。

三条线索指向工具链而非选手代码：1. 对照组用 gcc-13 + -march=rv64gc（默认），#2 用 gcc-14 + -march=rv64gcv_zba_zbb_zbc，codegen 不同 2. S2605 #3 的说明文件独立记载了同一现象：「options_settable_test: padding hole 用 offsetof 可移植排除（riscv64 上构造函数 store-merging 写 padding 导致计数不稳）」 3. 失败差值（126 - 102 = 24 字节）符合 padding 写入的特征，而非字段缺失

隔离实验结论（2026-09-13）：编译器产物，不归因于选手

用 #2 的工具链与编译选项（gcc-14 + -march=rv64gcv_zba_zbb_zbc）重编上游对照组源码（未含任何选手改动），只跑此用例：

```
[ RUN      ] OptionsSettableTest.ColumnFamilyOptionsAllFieldsSettable
unset_bytes_base  Which is: 95
NumUnsetBytes(...) Which is: 119
[  FAILED  ] OptionsSettableTest.ColumnFamilyOptionsAllFieldsSettable
exit=1
```

上游源码在同一工具链下同样失败，且偏差同为 24 字节。故 #2 与 #4 的该项失败均为 **gcc-14 codegen 写入结构体 padding** 所致，不归因于选手。日志：logs/ISOLATION-control-gcc14-rvv.{build,run}.log。

稳定性复跑：#2 的该用例连跑 5 次，结果完全一致（均 102 → 126，exit=1）—— **确定性失败，非随机 flaky**。即 #3 文档所称的「计数不稳」是指**跨编译配置**不稳，而非同一二进制跨次不稳，其描述与实测一致。

四、S2605 #4 —— 失败集合与 #2 同型，无独有偶

耗时 2,299 秒，测试跑满（Left: 0）。失败项：

用例	退出码	归因
range_locking_test ×16 项	132 (SIGILL)	上游固有（同对照组，rdcycle; 16 vs 14 为枚举中止时机差异，非新增）
options_settable_test ColumnFamilyOptionsAllFieldsSettable	134 (SIGABRT)	待定性（同 #2，见 §3）

一个已排除的疑点（记录以备查）

初见 #4 比对照组多出 RangeLockingTest.BasicLockEscalation、LockEscalationBarrier 两项失败，一度怀疑其分支基线并非赛题指定的 v11.1.1。经核实此疑点不成立：- 上游 v11.1.1 源码中这两个用例本就存在（range_locking_test.cc:256、:296）

· git describe：对照组 v11.1.1、#4 为

v11.1.1-11-g9ec8dc8e4（即 v11.1.1 之后 11 个提交，均为选手自身改动）——**基线版本正确**

差异仅源于两次运行中止时机不同导致的用例枚举不全。**不构成对 #4 的任何不利结论。**

与 #2 的交叉印证（对 options_settable_test 定性的强证据）

提交	编译器 / march	unset_bytes_base	实测值	差值
对照组	gcc-13 / rv64gc	—	—	通过
#2	gcc-14 / rv64gcv_zba_zbb_zbc	102	126	+24
#4	gcc-14 / rv64gcv (PORTABLE)	95	119	+24

两个互不相干的选手代码库，在 gcc-14 下出现完全相同的 24 字节偏差，而 gcc-13 的对照组通过。基数不同（102 / 95）是两个 fork 的结构体本身略有差异所致，**偏差恒为 24 字节**这一点只能由编译器写入 padding 解释。加上 #3 文档独立记载同一现象（§3），三方证据一致。

隔离实验已完成（上游源码 + gcc-14 + #2 的 march 下同样失败且偏差同为 24 字节），已证实仍不下最终结论**。

五、组委会侧问题（已修正，记录备查）

问题	影响	修正
板上未装 clang-format	#3 的 make check 在测试全通过后失败于 check-format，若照单全收会误判	2026-09-13 补装 18.1.3；#3 的正确性结论以测试阶段为准
pass/fail 计数表达式按 gtest 单体格式编写	汇总表 pass 列恒为 0（日志完好，不影响结论）	见下节，待统一重导
make check-format 会就地改写被测源码	RocksDB 的 format-diff.sh -c 在检查时自动向选手新增文件写入版权头，改动了 #3 工作树的 9 个文件（rvv-ci/*.cc、tools/b2_analyze.py）。评审绝不应改动选手代码	已 git checkout -- . 全部还原（改动数归 0），并核验 HEAD 仍为冻结 SHA fa3fd926（与 MANIFEST.tsv 一致）。后续如需 check-format，须先复制工作树再跑，或只用 --check 只读模式

check-format 结果（归 § 4.E 代码规范，不作正确性判定）

提交	结果	说明
#3	未通过	「Your change has unformatted code」；另有 9 个新增文件缺版权头
#2 / #4	未单独复跑	其 make check 在测试阶段即失败退出，未走到 check-format

注：此项仅供 § 4.E 「代码规范，酌情记录」参考，不影响 § 4.C 正确性结论。若要横向可比，须对三个提交在同一条件下补跑（且须用只读方式，见上表）。

六、已知的统计口径问题（不影响结论，待收尾修正）

本轮脚本中 pass/fail 的计数表达式按 gtest 单体输出格式编写（[PASSED]），而 RocksDB 的 make check 走 GNU parallel 汇总输出，格式不同，导致 makecheck.tsv 中 pass 列恒为 0。原始日志完整无损，失败清单以日志中 parallel joblog 的 Exitval 列为准（本报告第一节即由此得出）。为避免干扰正在运行的作业，计数逻辑不在运行中修改，待全部跑完后用统一解析脚本从日志重新导出。

第四部分 热点函数表

采样日期：2026-09-13 对应验收备忘录 § 5.2 「RVV 向量化占比——70% 类」的分母表

规则原文的分母（「数据库内核 / 核心函数 / 字符串核心 / 协议解析函数」）无客观边界，备忘录 § 5.2 改为热点覆盖评估：在验证平台用 perf 采样 memtier 压测下的标量版热点，取 Top 20 热点函数为分母，检查其中被 RVV 向量化（或其调用的核心路径被向量化）的比例，≥ 70% 为过。

一、采样方法

项 | 值 |

版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

|—|—| | 平台 | 蓝芯 LX5000（与基线、性能测试同一环境） | | 被采样二进制 | 组委会标量基线（Redis 8.8.0 / Memcached 1.6.42，默认 -march=rv64gc） | | 负载 | memtier base 场景（-t 4 -c 50 -d 32，ratio 1:10），S2603 另采 --pipeline=16 | | 采样器 | perf record -F 199 -g，用户

态采样 (perf_event_paranoid=2, 无需 root) || 采样窗口 | 压测启动后跳过 15s 预热, 稳态采样 60s || 绑核 | 服务端 0-7、压测端 16-31 (与基线一致) |

原始数据: LX5000 ~/rvspoc-perf/hotspot/*.data; 脚本 ~/rvspoc-perf/hotspot.sh。

归类口径 (下表「类别」列): - 自有代码: 被测项目自身的函数 (redis-server / memcached 符号)

系统调用/内核出口: read/write/sendmsg/epoll_pwait/vDSO——选手无法向量化

· 锁原语: pthread 互斥与取消点——选手无法向量化 - libc 字符串/内存: memchr/memcpy/memset/strcascmp——理论上可由 libc

侧向量化, 但不属选手改动范围 (除非选手自行替换实现) - 第三方库: libevent 等

二、S2603 Redis —— base 场景 Top 20

#	占比	DSO	函数	类别
1	9.67%	libc	__GI__libc_write	系统调用
2	6.71%	redis-server	processInputBuffer	自有代码 (协议解析)
3	6.38%	libc	read	系统调用
4	6.20%	redis-server	writeToClient	自有代码
5	5.59%	redis-server	readQueryFromClient	自有代码 (协议解析入口)
6	4.18%	redis-server	updateClientMemUsageAndBucket	自有代码
7	4.13%	vdso	__vdso_clock_gettime	内核出口
8	3.10%	redis-server	call	自有代码
9	2.86%	redis-server	dictFindLinkInternal	自有代码 (哈希查找)
10	2.68%	redis-server	handleClientsWithPendingWrites	自有代码
11	2.07%	redis-server	connSocketEventHandler	自有代码
12	1.97%	redis-server	processCommand	自有代码
13	1.78%	redis-server	resetClientInternal	自有代码
14	1.74%	redis-server	addReply	自有代码
15	1.64%	redis-server	aeMain	自有代码 (事件循环)
16	1.31%	libc	__strcascmp	libc 字符串
17	1.31%	redis-server	zmalloc_usable	自有代码 (分配器)
18	1.27%	redis-server	resetReusableQueryBuf	自有代码
19	1.27%	redis-server	siphash	自有代码 (哈希)
20	1.22%	redis-server	decrRefCount	自有代码

构成: 自有代码 16 项 (占 Top20 内 44.6 个百分点)、系统调用/vDSO 3 项 (20.2)、libc 字符串 1 项 (1.3)。

三、S2603 Redis — pipeline=16 场景 Top 20

管道场景压低了 syscall 占比，协议解析与哈希路径权重上升，更接近赛题关注点。

#	占比	DSO	函数	类别
1	8.16%	redis-server	processInputBuffer	自有代码（协议解析）
2	6.12%	redis-server	dictPrefetcherRun	自有代码（预取）
3	5.47%	redis-server	siphash	自有代码（哈希）
4	5.26%	redis-server	call	自有代码
5	4.85%	vdso	__vdso_clock_gettime	内核出口
6	3.87%	redis-server	dictFindLinkInternal	自有代码
7	3.50%	redis-server	zmalloc_usable	自有代码
8	2.84%	redis-server	resetClientInternal	自有代码
9	2.67%	redis-server	decrRefCount	自有代码
10	2.53%	redis-server	processCommand	自有代码
11	2.08%	libc	__GI__libc_write	系统调用
12	1.91%	libc	__strcasecmp	libc 字符串
13	1.89%	libc	memchr	libc 字符串（RESP 分隔符扫描）
14	1.81%	redis-server	readQueryFromClient	自有代码
15	1.69%	redis-server	getKeysUsingKeySpecs	自有代码
16	1.61%	redis-server	lookupKey	自有代码
17	1.57%	redis-server	sdsnewplacement	自有代码（SDS）
18	1.54%	libc	memcpy	libc 内存
19	1.50%	redis-server	ACLCheckAllUserCommandPerm	自有代码
20	1.47%	libc	read	系统调用

构成：自有代码 14 项（48.6 个百分点）、系统调用/vDSO 3 项（8.4）、libc 字符串/内存 3 项（5.3）。

四、S2604 Memcached — base 场景 Top 20

#	占比	DSO	函数	类别
1	14.26%	libc	pthread_mutex_lock	锁原语
2	8.56%	libc	__libc_sendmsg	系统调用
3	7.06%	libc	read	系统调用
4	5.66%	libc	pthread_mutex_unlock_usercnt	锁原语
5	4.78%	memcached	drive_machine	自有代码（状态机）
6	4.08%	libc	epoll_pwait	系统调用
7	2.92%	libevent	evmap_io_active_	第三方库

8	2.37%	memcached	_mcmc_tokenize_meta	自有代码（协议分词）
9	2.26%	libc	pthread_enable_asynccancel	锁原语/取消点
10	2.22%	memcached	conn_set_state	自有代码
11	1.91%	memcached	process_request	自有代码（协议）
12	1.86%	memcached	assoc_find	自有代码（哈希查找）
13	1.84%	memcached	process_command_ascii	自有代码（协议）
14	1.77%	libc	memchr	libc 字符串
15	1.71%	libc	pthread_disable_asynccancel	锁原语/取消点
16	1.69%	libevent	event_base_loop	第三方库
17	1.49%	libc	pthread_mutex_unlock	锁原语
18	1.44%	memcached	MurmurHash3_x86_32	自有代码（哈希）
19	1.39%	memcached	resp_allocate	自有代码
20	1.38%	libc	memset	libc 内存

构成：自有代码仅 8 项（17.8 个百分点）、锁原语 5 项（25.4）、系统调用 3 项（19.7）、第三方库 2 项（4.6）、libc 字符串/内存 2 项（3.2）。

五、这张表说明的问题（与性能结果互相印证）

1. **两题的热点主体都不可向量化。**S2604 的第 1 位是 pthread_mutex_lock（14.3%），锁原语加系统调用合计超过 45%；S2603 base 场景的 read/write/vDSO 合计 20%。向量化能触及的只有协议解析、哈希、SDS 这一部分。
2. **这解释了性能测试结果：**S2603/S2604 全部提交对基线的差异都在 $\pm 4\%$ 以内（见《LX5000 卷》第二部分）。即便协议解析被完全向量化，按 Amdahl 定律其上限也远达不到 +30%——瓶颈在网络 I/O 与锁竞争。
3. **管道场景是唯一有空间的方向：**pipeline=16 下自有代码占比升至 48.6%，syscall 降至 8.4%。实测中唯一有两位数收益的也正是此场景（条件组 S2603 #6 的 pipe64 +16.5%）。

六、一个必须先定的口径问题（待决策 D10）

§ 5.2 写「取 Top 20 热点函数为分母，检查其中被 RVV 向量化的比例 $\geq 70\%$ 为过」。但按字面执行，分母里含 pthread_mutex_lock、read、sendmsg、epoll_pwait、vDSO —— **这些选手无论如何都无法向量化**。S2604 的 Top 20 中自有代码仅 8 项（40%），即使选手把自有代码全部向量化，覆盖率上限也只有 40%，**永远不可能达到 70%**。

三种可选口径：

- **a) 字面执行：**Top 20 原样为分母 $\rightarrow$ S2604 全员必然不过门槛，S2603 上限约 70–80%（base 场景 16/20）
- **b) 剔除不可向量化项：**分母只取「自有代码」函数（S2603 base 16 项 / pipe16 14 项、S2604 8 项），再算 70% —— **建议采用**，符合规则「核心函数 / 字符串核心 / 协议解析函数」的原意
- **c) 按占比加权：**以自有代码热点的时间占比而非函数个数计算

建议 b，并在评审记录中写明分母清单（本表第二至四节即为清单）。此为门槛项口径，一旦选定将直接决定 S2603/S2604 两题是否有提交能过 RVV 覆盖门槛，**须评判组定夺**。

第五部分 组委会标量基线

版本 v1.0 发布日期 2026-09-22 本包 17 份文件共用同一版本号

生成日期：2026-09-03 对应验收备忘录 § 2「组委会自测基线」、§ 4D「性能」

本报告给出 S2603 / S2604 / S2605 三题在蓝芯 LX5000 上的**组委会自测标量基线**。选手性能是否达到「vs 组委会标量基线 $\geq +30\%$ 」，以本报告数字为唯一判定依据；**不采用选手自行提交的基线数**。

一、验证环境（定版）

项	值
平台	蓝芯 LX5000, 32 hart, 251 GiB RAM, 1.8 TB 存储
ISA	rv64imafdcvh_zicbom..._zvbb_zvkned... (含 V 扩展, 基线构建 不 启用)
mvendorid / marchid	0x489 / 0x8000000000000109
OS	Ubuntu 24.04.4 LTS
内核	6.18.3+ SMP PREEMPT_DYNAMIC
编译器	gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
glibc	2.39-0ubuntu8.8

环境完整快照：`baseline/results/env-20260901T141057Z/` (uname、cpuinfo、依赖包列表等)。

基线定义：上游 pinned 版本源码 + 发行版默认工具链构建（默认 `-march=rv64gc`，**无 V 扩展**），不加任何 `-march`、`-mrvv` 参数。这是「标量基线」的口径。

组件	版本
Redis	8.8.0 (MALLOC=jemalloc)
Memcached	1.6.42
memtier_benchmark	2.4.1
RocksDB	v11.1.1 (DEBUG_LEVEL=0, db_bench)

二、测试方法（选手复现验证时必须使用同一套参数）

- **绑核**：被测服务 CPU 0~7，压测端 CPU 16~31 (taskset)，避免互相干扰
- **memtier**：`--test-time=60 --key-maximum=1000000 --random-data`，GET:SET = 10:1（写重场景 1:1）
- **db_bench**：`--num=5000000 --value_size=100 --threads=8 --compression_type=none`，每轮全新库
- **每场景 3 轮，取中位数**；轮间冷却 30 秒、场景间冷却 90 秒（避免热积累与状态残留）

复现脚本：baseline/ (00_env_snapshot.sh ... 30_summarize.py)，原始输出全部留档。

三、基线数据

3.1 S2603 — Redis 8.8.0 标量 (results/s2603-20260902T152013Z)

场景	参数	Totals QPS	P99 (ms)
基础	4 线程 / 50 连接 / 32B	41,827	9.41
高并发	8 线程 / 200 连接 / 32B	29,752	104.45
写重	4/50, ratio 1:1	41,162	9.47
高 Value 4K	4/50 / 4096B	42,048	9.34
高 Value 16K	4/50 / 16384B	36,035	49.15
管道 16	4/50, pipeline=16	412,904	14.78
管道 64	4/50, pipeline=64	746,708	23.17
AOF 持久化	appendfsync everysec	41,524	9.41

3.2 S2604 — Memcached 1.6.42 标量 (results/s2604-20260903T021121Z)

场景	参数	Totals QPS	P99 (ms)
基础	4/50 / 32B	109,621	2.69
高并发	8/200 / 32B	150,285	38.14
写重	4/50 / 1024B, ratio 1:1	105,816	2.81
高 Value 4K	4/50 / 4096B	103,932	2.85
管道 16	4/50, pipeline=16	677,363	10.18

服务端启动参数：-t 4 -m 4096 -c 4096。-c 4096 是必需的：memcached 默认 maxconns=1024，低于高并发场景所需的 1600 连接，会导致连接被拒、memtier 报 response parsing failed 且 QPS 归零（首次运行 s2604-20260902T160443Z 即因此作废）。选手验证时须使用相同的 -c 4096，否则高并发场景不可比。

3.3 S2605 — RocksDB v11.1.1 标量 (results/s2605-20260902T163232Z)

项目	ops/sec	micros/op	备注
fillrandom	244,774	32.59	必测
readrandom	292,356	26.99	必测
seekrandom	141,371	55.68	必测 (--seek_nexts=10)
fillseq	260,890	30.66	参考
overwrite	243,093	32.86	参考
readseq	8,887,222	0.90	参考，见下方警示

判定口径：+30% 门槛只用必测三项（fillrandom / readrandom / seekrandom）。readseq 达 888 万 ops/sec，是随机读的 30 倍——顺序扫描几乎全部命中 block cache，该指标对向量化优化不敏感，若选手以此类指标主张收益，不予采信。

四、测量稳定性

3 轮之间的离散度（全部 19 个场景）：

- 变异系数 CV ≤ 2.4%，轮间极差 ≤ 4.7%
- 最稳定：s2604/highval4k（CV 0.3%）；波动最大：s2605/readseq（CV 2.4%）

对判定的意义：测量噪声比 +30% 门槛小一个数量级，因此「是否达到 +30%」不会因噪声产生歧义。但需注意跨会话波动略大于会话内波动——S2604 基础场景首次测得 112,505 QPS、重跑测得 109,621 QPS，相差 2.6%。故：

- 选手成绩与基线的比较，只与本报告数字比，不与选手自测数字交叉比较
- 若选手成绩落在 +28% ~ +32% 的边界带，应重跑复核后再定论

五、留档清单

内容	路径
复现脚本	baseline/*.sh、baseline/30_summarize.py
环境快照	baseline/results/env-20260901T141057Z/
原始输出（19 场景 × 3 轮）	baseline/results/s260{3,4,5}-*/
汇总	各目录下 summary.json / summary.md
作废数据（S2604 首次）	baseline/results/s2604-20260902T160443Z/（高并发场景无效，保留备查）

六、尚未完成

- S2602 精度参考：不在本报告范围（S2602 性能门槛是绝对值 110ms，无需标量基线）。x86 参考环境已就绪（六个模型下载校验完毕、评测脚本已写），待测试集定版后出数。